

Method Name

Sunday, 17 June 2024 15:53

Query Methods

<https://docs.spring.io/spring-data/jpa/reference/jpa/query-methods.html>

- JPA can generate queries **automatically based on the method names** you define in a repository interface.
- This is one of the **most convenient features** because it allows you to write clean, self-explanatory code without dealing with any SQL or JPQL directly.

How It Works:

- **Method Naming Convention:** The method's name follows a particular pattern: `findBy`, `readBy`, or `queryBy` followed by the entity's properties. Spring parses this name, translates it into the corresponding SQL/JPQL, and executes the query.
- **Supported Keywords:** You can use keywords such as `And`, `Or`, `Between`, `LessThan`, `Like`, and more. This helps in creating queries for multiple conditions.

Example:

```
public interface UserRepository extends JpaRepository<User, Long> {  
    // Find users by last name  
    List<User> findByLastName(String lastName);  
    // Find users by age range  
    List<User> findByAgeBetween(Integer startAge, Integer endAge);  
    // Find user by first name and last name  
    Optional<User> findByFirstNameAndLastName(String firstName, String lastName);  
}
```

```
@Entity  
public class User {  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    private String firstName;  
    private String lastName;  
    private Integer age;  
  
    // getters and setters  
}
```

Advantages:

- **Simplicity:** No need to write boilerplate code for common queries.
- **Maintainability:** The intent is expressed in the method name, making the code self-documenting.
- **Type-Safety:** Since the method signature is well-defined, you get compile-time checking.

Limitations:

- **Complex Queries:** When queries become more intricate (e.g., involving joins, grouping, or subqueries), method names can become unwieldy or may not support the required complexity.

Keyword	Sample
Distinct	<code>findDistinctByLastNameAndFirstName</code>
And	<code>findByLastNameAndFirstName</code>
Or	<code>findByLastNameOrFirstName</code>
Is, Equals	<code>findByFirstName, findByFirstnames, findByFirstNameEquals</code>
Between	<code>findByStartDateBetween</code>
LessThan	<code>findByAgeLessThan</code>
LessThanEqual	<code>findByAgeLessThanEqual</code>
GreaterThan	<code>findByAgeGreaterThan</code>
GreaterThanEqual	<code>findByAgeGreaterThanEqual</code>
After	<code>findByStartDateAfter</code>
Before	<code>findByStartDateBefore</code>
IsNull, Null	<code>findByAgeIsNull</code>
IsNotNull, NotNull	<code>findByAgeIsNotNull</code>
Like	<code>findByFirstNameLike</code>
NotLike	<code>findByFirstNameNotLike</code>
StartingWith	<code>findByFirstNameStartingWith</code>
EndingWith	<code>findByFirstNameEndingWith</code>
Containing	<code>findByFirstNameContaining</code>
OrderBy	<code>findByAgeOrderByLastNameDesc</code>
Not	<code>findByLastNameNot</code>
In	<code>findByAgeIn(Collection<Age> ages)</code>
NotIn	<code>findByAgeNotIn(Collection<Age> ages)</code>
True	<code>findByActiveTrue()</code>
False	<code>findByActiveFalse()</code>
IgnoreCase	<code>findByFirstNameIgnoreCase</code>

